ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO



Industrial IoT: Predictive Maintenance E2E Pipeline: Practical Demonstration and Conclusions

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departmento de Engenharia Informática

Industrial IoT: Predictive Maintenance E2E Pipeline

Practical Demonstration

Demo Architecture and E2E Topology

- **Scenario:** Industrial motor predictive maintenance (temperature and vibration monitoring).
- **Data Flow Pipeline:**
  1. **Mosquitto (MQTT):** Telemetry ingestion.
  2. **IoT Agent JSON:** Protocol translation to NGSI-v2.
  3. **FIWARE Orion:** Context Broker for real-time state management.
  4. **Persistence Connector / Cygnus:** Event forwarding.
  5. **TimescaleDB:** Time-series historical storage.
  6. **Grafana:** Visual dashboards and alerting.

Step 1: Project Structure and Docker Compose

Directory Layout

```
industrial-iot-demo/
|-- docker-compose.yml
|-- .env
|-- mosquitto/
|   `-- mosquitto.conf
|-- iot-agent/
|   `-- config.js
`-- scripts/
    `-- simulate-telemetry.sh
```

Step 1.1: Docker Compose (Context Broker)

Infrastructure configuration snippet for FIWARE core services

```
# DOCKERFILE STARTS HERE

services:
  orion:
    image: fiware/orion:3.9.0
    ports:
      - "1026:1026"
    command: -logLevel DEBUG

# (CONTINUES)
```

Step 1.1: Docker Compose (IoT Agent)

Infrastructure configuration snippet for FIWARE core services (cont.)

```
  iot-agent:
    image: fiware/iotagent-json:1.27.0
    ports:
      - "4041:4041"
    environment:
      - IOTA_CENTRAL_CONFIGURATION=true
      - IOTA_MQTT_HOST=mosquitto
      - IOTA_MQTT_PORT=1883
      - IOTA_ORION_HOST=orion
      - IOTA_ORION_PORT=1026

# (CONTINUES)
```

Step 1.2: Docker Compose (Broker, DB and Dashboard)

Infrastructure configuration snippet for messaging (cont.)

```
  mosquitto:
    image: eclipse-mosquitto:2.0
    ports:
      - "1883:1883"
      - "9001:9001"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf

# (CONTINUES)
```

Step 1.2: Docker Compose (Broker, DB and Dashboard)

Infrastructure configuration snippet for persistence (cont.)

```
  timescaledb:
    image: timescale/timescaledb:latest-pg14
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_PASSWORD=secret

# (CONTINUES)
```

Step 1.2: Docker Compose (Broker, DB and Dashboard)

Infrastructure configuration snippet for dashboard (cont.)

```
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"

# DOCKERFILE ENDS HERE
```

Step 1.3: Project Structure and Docker Compose (cont.)

Start infrastructure services

```
$ docker compose config
$ docker compose up -d
$ docker compose ps
```

Step 2: Device Provisioning in IoT Agent

Register the motor entity and map MQTT attributes using ‘curl‘

```
$ curl -i -X POST "http://localhost:4041/iot/devices" \
  -H 'fiware-service: motorlab' \
  -H 'fiware-servicepath: /' \
  -H 'content-type: application/json' \
  -d '{
    "devices": [
      {
        "device_id": "motor001",
        "entity_name": "Machine001",
        "entity_type": "IndustrialMachine",
        "protocol": "IoTAgent-JSON",
        "transport": "MQTT",
        "attributes": [
          {"object_id": "t", "name": "temperature", "type": "Number"},
          {"object_id": "v", "name": "vibration", "type": "Number"}
        ]
      }
    ]
  }'
```

Step 3: Simulating MQTT Telemetry

Publish real-time operational data from the motor

```
# Normal operating state
$ mosquitto_pub -h localhost -p 1883 \
  -t '/json/1234/motor001/attrs' \
  -m '{"t": 72.5, "v": 3.8}'

# Anomaly simulation (high temperature and vibration)
$ mosquitto_pub -h localhost -p 1883 \
  -t '/json/1234/motor001/attrs' \
  -m '{"t": 92.0, "v": 9.4}'
```

Step 4: Verifying Context in Orion

Query the Orion Context Broker via REST API to verify current entity state

```
$ curl -sS 'http://localhost:1026/v2/entities/Machine001' \
  -H 'fiware-service: motorlab' \
  -H 'fiware-servicepath: /' \
  | python -m json.tool
```

Expected output shows updated attributes: ‘temperature‘ and ‘vibration‘.

Step 5: TimescaleDB Historical Storage

Step 5.1: TimescaleDB Schema Definition

Orion manages current context, but time-series analysis requires a specialized database. Create the base telemetry table:

```
CREATE TABLE machine_telemetry (
    time        TIMESTAMPTZ NOT NULL,
    machine_id  TEXT NOT NULL,
    temperature DOUBLE PRECISION,
    vibration   DOUBLE PRECISION
);
```

This defines the structure where each timestamped record of the industrial machine will be securely logged.

Step 5: TimescaleDB Historical Storage

Step 5.2: Converting to a TimescaleDB Hypertable

Transform the standard PostgreSQL table into an optimized time-series hypertable:

```
SELECT create_hypertable(
    'machine_telemetry',
    by_range('time'),
    if_not_exists => TRUE
);
```

This function automatically partitions data by time blocks, enabling high-performance historical queries and efficient storage compression.

Step 5: TimescaleDB Historical Storage

Step 5.3: Executing SQL Live via Docker

How to run the SQL commands directly during the workshop using the terminal:

```
$ docker exec -it timescaledb psql -U postgres -c "
CREATE TABLE machine_telemetry (
    time TIMESTAMPTZ NOT NULL,
    machine_id TEXT NOT NULL,
    temperature DOUBLE PRECISION,
    vibration DOUBLE PRECISION
);
SELECT create_hypertable('machine_telemetry',
       by_range('time'), if_not_exists => TRUE);"
```

Alternative: Connect any external SQL GUI client (like DBeaver or pgAdmin) to `localhost:5432` using user `postgres`.

Step 5: TimescaleDB Historical Storage

Step 5.4: Configuring Orion Subscriptions

Configure the Context Broker to forward attribute updates to the persistence connector:

```
curl -i -X POST 'http://localhost:1026/v2/subscriptions' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: motorlab' \
  -d '{
    "description": "Persist motor telemetry",
    "subject": {
      "entities": [{"id": "Machine001", "type": "IndustrialMachine"}]
    },
    "notification": {
      "http": { "url": "http://telemetry-connector:8080/notify" },
      "attrs": ["temperature", "vibration"]
    }
  }'
```

Step 6: Grafana Dashboards and Alerting

- **Data Source Connection:** PostgreSQL / TimescaleDB integration.
- **Panels Configured:**
  - Temperature time series graph (`$__timeFilter`).
  - Vibration trends over time.
  - Last measured state (Stat / Table view).
- **Alert Rules:** Threshold triggers for overheating or mechanical vibration anomalies.

Step 6.1: Grafana Dashboards and SQL Queries

Connect Grafana to TimescaleDB and configure standard panels using native time filters:

```sql
-- Panel 1: Temperature Over Time
SELECT
    time AS "time",
    temperature AS "Temperature (C)"
FROM machine_telemetry
WHERE $__timeFilter(time) AND machine_id = 'motor001'
ORDER BY time;

-- Panel 3: Last Measured State (Stat / Table view)
SELECT time, temperature, vibration
FROM machine_telemetry
WHERE machine_id = 'motor001'
ORDER BY time DESC LIMIT 1;

```

Step 6.2: Sample Telemetry Data Visualized

Example rows stored in TimescaleDB and rendered in Grafana tables:

```
-------------------------------------------------------------------
time                  | machine_id | temperature | vibration
-------------------------------------------------------------------
2026-10-13 18:30:00   | motor001   | 71.2        | 3.5
2026-10-13 18:30:05   | motor001   | 72.5        | 3.8  (Normal)
2026-10-13 18:30:10   | motor001   | 78.9        | 5.2
2026-10-13 18:30:15   | motor001   | 86.2        | 7.1  (Warning)
2026-10-13 18:30:20   | motor001   | 92.0        | 9.4  (Anomaly!)
-------------------------------------------------------------------

```

High-frequency polling tracks continuous mechanical degradation over time.

ISEP

Step 6.3: Grafana Alert Rules and Thresholds

Define predictive maintenance alert rules directly in Grafana to spot anomalies:

- **Overheating Threshold:** Triggered if temperature exceeds 90.0 °C.
- **Vibration Anomaly:** Triggered if vibration exceeds 8.0 (indicating imbalance or wear).

```
-- Alert evaluation query example
SELECT
    CASE WHEN temperature > 90.0 OR vibration > 8.0
         THEN 1 ELSE 0 END AS alert_status
FROM machine_telemetry
WHERE machine_id = 'motor001'
ORDER BY time DESC LIMIT 1;

```

Synthesis and Production Challenges

- **Key Takeaways:**
  - Effective decoupling via MQTT Broker and NGSI context management.
  - Separation of real-time state (Orion) from time-series storage (TimescaleDB).
- **Production Realities:**
  - **Scalability:** Subscription management overhead and database indexing.
  - **Security:** OAuth2, TLS encryption, and PEP Proxies.
  - **Edge Computing:** Local data retention, offline operation, and store-and-forward mechanisms.